

Projet Intégrateur : PacMan en réseau

Thomas Coutant | Benoît Litampha | Auriane Peter

Université Marie & Louis Pasteur, UFR Sciences et Techniques

Licence CMI Informatique – 3^{ème} année
2025–2026

Encadrant : Julien Bernard



Plan

Présentation et organisation

Serveur

Client

Réseau

Démonstration

Introduction

Contexte

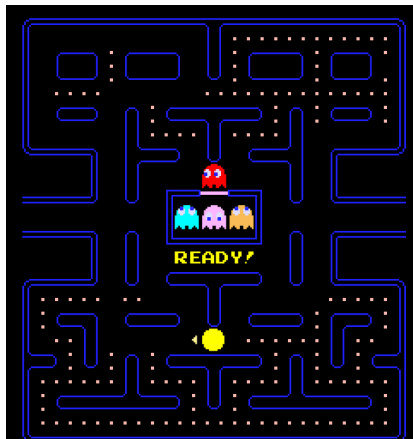
- Années 1980
- Jeu d'arcade

Gameplay

- Labyrinthe
- Pac-gommes
- Fantômes
- Mode chasse (power-up)

Projet

- Jeu réseau
- Génération procédurale
- IA des fantômes



Pac-Man (original)

Adaptation du projet :

- 1 joueur Pac-Man
- Plusieurs joueurs Fantômes
- Fantômes contrôlés par **IA** si joueurs insuffisants
- Cabane comme lieu d'apparition des fantômes

Déroulement d'une partie :

Connexion > Liste des Salons > Salon > Jeu > Fin de jeu

Plan

Présentation et organisation

Serveur

Client

Réseau

Démonstration

- **Git**
- **C++** avec le framework **GameDev**
- **Réunions régulières** : toutes les 1 à 2 semaines
- **Communication quotidienne** avec **Discord**



■ Auriane Peter

- Développement du client
- Interface et interactions joueur

■ Benoît Litampha

- Communication client–serveur
- Support sur client et serveur

■ Thomas Coutant

- Développement du serveur
- Conception de la logique du jeu

Plan

Présentation et organisation

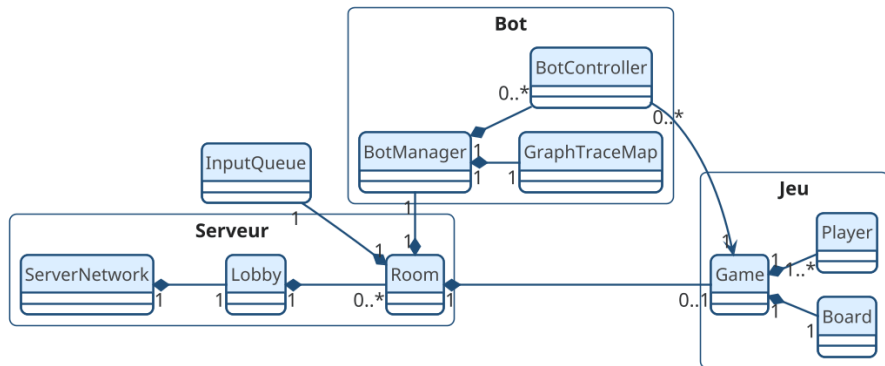
Serveur

Client

Réseau

Démonstration

Architecture



Client → Serveur → Hall → Salon → Game

Architecture générale

Architecture

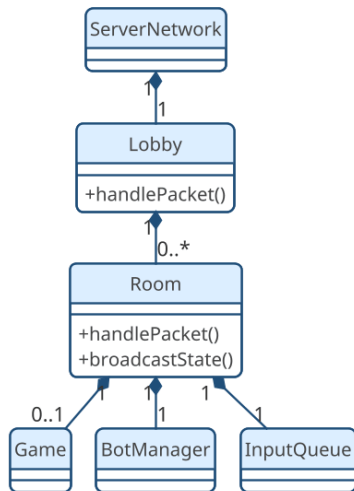
- Organisation **modulaire**
- Séparation des responsabilités
- Lisibilité et maintenabilité

Couches

- Réseau : Serveur
- Organisation : Hall
- Partie : Salon
- Logique : Jeu

Rôle central

- Coordination des composants
- Gestion des joueurs
- Synchronisation des états



Boucle de jeu

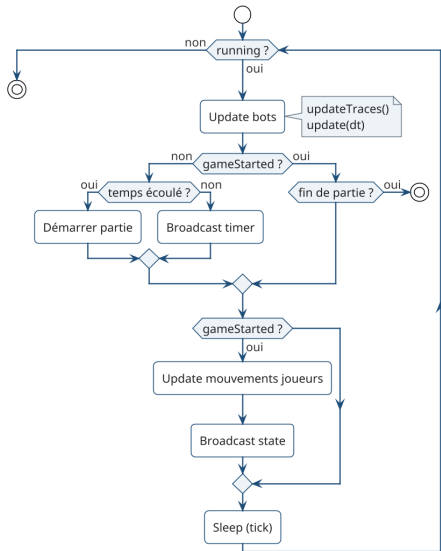
- Serveur **autoritaire**
- Affichage **côté client**
- Anti-triche et synchronisation

Tick serveur

- Mise à jour **périodique**
- Thread serveur :
 - Positions
 - Collisions
 - Scores et timers

États

- Pré-jeu
- Partie en cours
- Fin de partie



IA des fantômes - Principe

Objectif

- Adaptation au nombre de joueurs
- Intelligence collective

Inspiration : fourmis

- Phéromones
- Communication indirecte
- Suivi de traces

Implémentation

- Traces sur le plateau
- Paramètres :
 - Intensité (décroissance)
 - Propriétaire
 - Couleur

```
struct Trace {  
    TraceType type;  
    uint32_t ownerId;  
    float intensity;  
};
```

Rouge : Pac-Man

| Bleu : fantômes

IA des fantômes - Prise de décision

Score

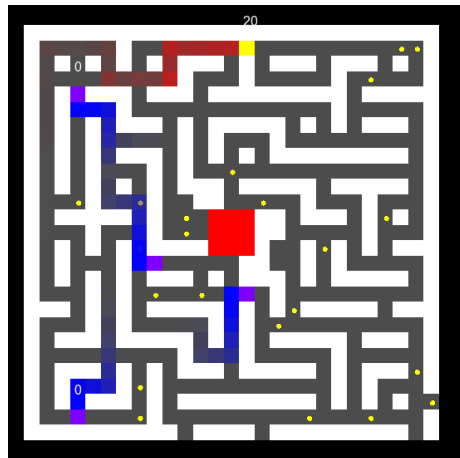
$$\text{Score} = \text{Attraction} - \text{Répulsions}$$

Influences

- Pac-Man -> Attraction
- Fantômes -> Répulsion
- Répulsion ++ pour lui-même
- Vision -> priorité Pac-Man
- Tracking -> BFS

Intégration serveur

- Bots = joueurs
- Envoi d'inputs
- InputQueue -> Salon



Génération procédurale du plateau

Algorithme

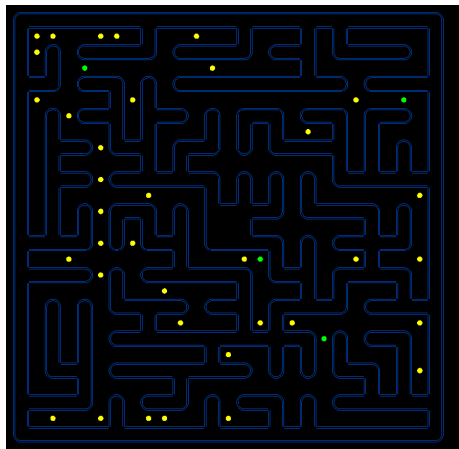
- Variante de Prim

Étapes

- Plateau = murs
- Case de départ
- Liste des murs adjacents
- Ouverture aléatoire

Propriétés

- Connexe
- Aucune zone isolée

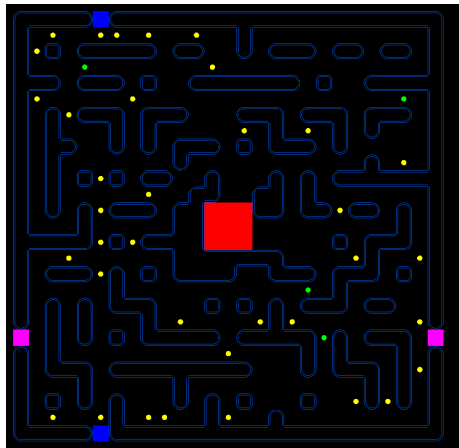


Contraintes

- Zone centrale : hut 3×3
- Accès $\rightarrow$ coins de départ

Améliorations

- Cycles
- Moins de culs-de-sac
- Portails latéraux



Plan

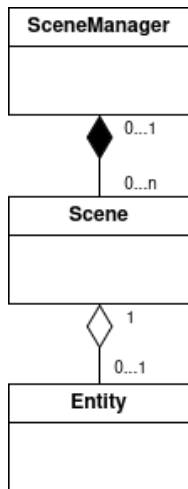
Présentation et organisation

Serveur

Client

Réseau

Démonstration



Fonctionnement du SceneManager

Le SceneManager gère quelle scène s'affiche puis :

- Chaque étape de jeu : **une scène**
- La scène se **gère elle même**
- **Séparation** entre logique / rendu

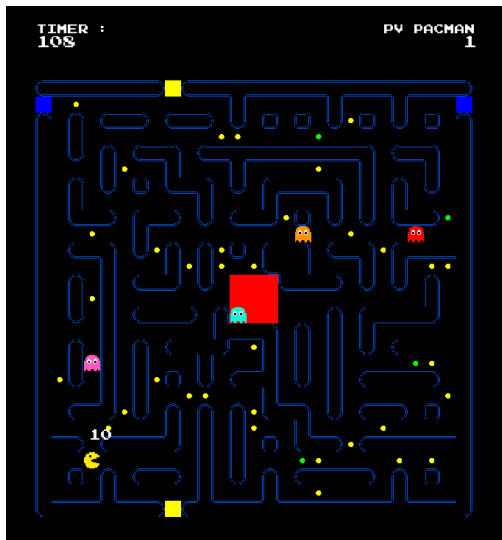
Met aussi en place le réseau

Système d'Événements :

- Mapping clavier -> actions (up, down, left, right)
- Passage de la souris
- Clics de la souris
- Redimensionnement de la fenêtre

Une partie est **envoyé au serveur**





Les scènes envoient les messages au serveur

Deux modèles :

- événementiel (menus, hall)
- périodique (mouvements en jeu)

Principe d'envoi est **le même**

L'envoi transmet l'intention du client au serveur

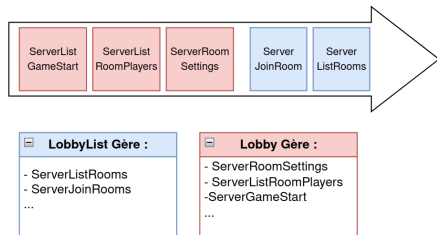
Reception

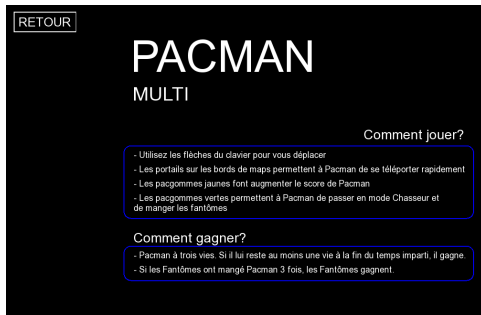
Réception dans un **Thread dédié**

Paquets stockés dans une file
partagée :

- Scène consomme paquets **selon leur type**
- Gère seulement ce **qui les concerne**

La scène se met à jour en fonction de
ce qu'elle a reçu





Chaque scène a une entité

Les entités ont **peu de travail** :

- Traduit la logique en **éléments graphiques concrets**
- Interagit directement avec le client : **clics client** -> **scène**

Plan

Présentation et organisation

Serveur

Client

Réseau

Démonstration

Communication avec socket

- Choix entre UDP et TCP : TCP choisit pour sa fiabilité

Classes Spécialisées

- **ServerNetwork** et **ClientNetwork** pour la réception des paquets

Sérialisation/désérialisation

- Conversion en suite d'octets pour l'envoi, reconstruction pour la réception

Multi-Threading

- Code de **ServerNetwork** et **ClientNetwork** exécutés en parallèle de la boucle principale

Structures communes

- Utilisées à la fois par le client et le serveur
- Peuvent être générées dynamiquement par le serveur

Structures de Communication

- Utilisées pour les échanges réseaux
- Un champ `gf::id` pour les identifier lors de la désérialisation
- Nomenclature spécifique :
 - *ServerXXX* = envoyée par le serveur
 - *ClientXXX* = envoyée par le client

```
struct PlayerData
{
    uint32_t id;
    float x, y;
    uint32_t color;
    std::string name;
    PlayerRole role;
    int score;
    bool ready = false;
    unsigned int hp = 0;
};
```

```
struct ServerListRoomPlayers
{
    static constexpr gf::Id type =
        "ServerListRoomPlayers"
        _id;
    std::vector<PlayerData>
        players;
};
```

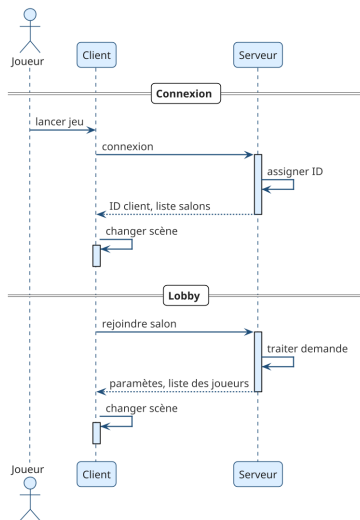

Structure des échanges

Connexion

- **Client** : Demande de connexion
- **Serveur** : Assignment d'un id et renvoi de l'id + liste des salons

Lobby

- **Client** : Demande de création/connexion à un salon
- **Serveur** : Traitement et envoi de la réponse et **broadcast** de la liste des joueurs/paramètres



Structure des échanges

Salon

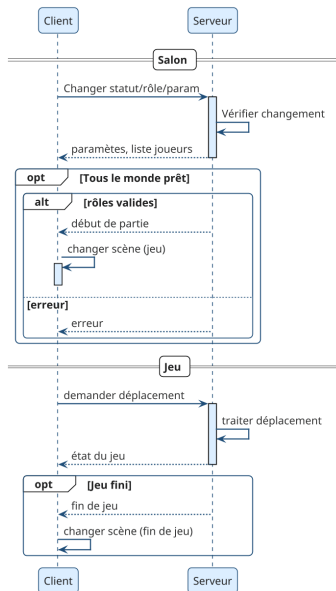
- **Client** : Demande de changement de rôle/status/paramètres
- **Serveur** : Traitement et **broadcast** de la liste des joueurs/paramètres

Si tous les joueurs son prêt : **Broadcast** début de partie ou erreur

En jeu

- **Client** : Demande de déplacement
- **Serveur** : Vérifie la validité, gère les conséquences puis **broadcast** de l'état du jeu

Si fin de jeu : **Broadcast** de la fin du jeu et de la raison



Objectif principal atteint

- Communication client/serveur
- Intelligence artificielle des fantômes
- Génération procédural de la carte

Aquisition et renforcement des compétences

- Programmation réseau, multi-threadé
- Modélisation des systèmes
- Travail en équipe

Perspectives

- Plusieurs rôles fantômes
- Intelligence artificielle pour Pac-Man
- Chambre privée

Plan

Présentation et organisation

Serveur

Client

Réseau

Démonstration

**Merci de votre attention.
Des question ?**